

# BYOP Project Report

## Movie Recommendation System

Content-Based Filtering using TF-IDF, Cosine Similarity, and K-Means Clustering

---

|           |                                      |
|-----------|--------------------------------------|
| Author    | ANURAG TIWARY                        |
| Course    | Fundamentals of AI and ML            |
| Platform  | VITyarthi                            |
| Reg. no.  | 25MIM10209                           |
| Technique | TF-IDF + Cosine Similarity + K-Means |

# 1. Introduction

Recommendation systems are among the most widely used applications of Machine Learning in the real world. Services like Netflix, Amazon, and Spotify all rely on recommendation engines to keep users engaged by showing them content they are likely to enjoy. Understanding how these systems work from the ground up is therefore both practically relevant and intellectually rewarding.

This project builds a content-based Movie Recommendation System that uses TF-IDF Vectorization and Cosine Similarity to recommend movies similar to one the user has already watched and enjoyed. An additional layer of K-Means Clustering groups movies into thematic clusters, demonstrating unsupervised learning in practice.

## 1.1 Motivation

The problem of deciding 'what to watch next' is one that nearly everyone faces. Instead of relying on a streaming service's opaque recommendation algorithm, this project builds a transparent, explainable recommender from scratch. The entire logic — from feature extraction to similarity computation — is visible and understandable, which aligns perfectly with the learning goals of this course.

## 1.2 Objectives

- Build a content-based recommender using TF-IDF and Cosine Similarity
- Return the top 5 most similar movies for any given input movie
- Apply K-Means Clustering to group movies into thematic clusters
- Visualize the similarity structure of the dataset using heatmaps and bar charts

# 2. Dataset

A custom dataset of 51 popular movies was created manually with metadata across 5 fields. Real-world datasets like TMDb and MovieLens could be used for future extensions, but for this beginner-level project, a hand-crafted dataset ensures full control over content quality and feature relevance.

## 2.1 Dataset Structure

| Field       | Description               | Example                    |
|-------------|---------------------------|----------------------------|
| title       | Movie name                | Inception                  |
| genre       | Primary genre(s)          | Sci-Fi Thriller            |
| director    | Director's name           | Christopher Nolan          |
| keywords    | Key thematic words        | dream heist memory reality |
| description | One-sentence plot summary | A thief enters dreams...   |

## 2.2 Genre Distribution

The dataset covers 12 primary genres with a reasonably balanced distribution. Sci-Fi and Animation are the most represented, with 8 movies each, followed by Action (7) and Crime, Horror, and Fantasy (4 each).

## 2.3 Combined Feature

All five fields are concatenated into a single 'combined' text field per movie. This combined representation serves as the input to the TF-IDF vectorizer, allowing the model to consider all aspects of a movie simultaneously when computing similarity.

# 3. Approach and Methodology

## 3.1 TF-IDF Vectorization

TF-IDF (Term Frequency–Inverse Document Frequency) is a technique for representing text as a numerical vector. It assigns higher weights to words that are frequent in a particular movie's description but rare across the entire dataset. This helps distinguish movies based on their unique characteristics rather than common words.

The vectorizer was configured with unigrams and bigrams (`ngram_range=(1,2)`) and English stop words removed. The resulting matrix has shape 51 x 1096, meaning 51 movies described by 1096 unique terms.

## 3.2 Cosine Similarity

Cosine Similarity measures the angle between two TF-IDF vectors. A similarity of 1.0 means the two movies are identical in terms of their feature representation; a score of 0.0 means they share no common relevant terms. The full 51x51 similarity matrix is computed in one step using scikit-learn's `cosine_similarity` function.

## 3.3 Recommendation Logic

Given an input movie title, the system looks up its index in the dataset, retrieves its row from the cosine similarity matrix, sorts all other movies by their similarity score in descending order, and returns the top N results. This is done in pure Python with no iterative model training — the 'model' is the precomputed similarity matrix itself.

## 3.4 K-Means Clustering

K-Means clustering was applied to the TF-IDF matrix with  $k=6$  to automatically discover thematic groupings in the movie dataset. This demonstrates unsupervised learning and validates that the TF-IDF representation captures meaningful semantic relationships between movies.

# 4. Results

## 4.1 Sample Recommendations

| Input Movie   | Top Recommendation | Similarity Score |
|---------------|--------------------|------------------|
| Inception     | Interstellar       | 0.1079           |
| The Godfather | Goodfellas         | 0.2270           |
| Toy Story     | Finding Nemo       | 0.1181           |
| Get Out       | Us                 | 0.3516           |
| Interstellar  | Gravity            | 0.1795           |

The recommendations are logically sound: The Godfather recommends Goodfellas (both gangster crime dramas), Get Out recommends Us (both Jordan Peele horror thrillers), and Interstellar recommends Gravity (both space survival Sci-Fi films). This validates that the TF-IDF + Cosine Similarity approach captures content similarity effectively.

### 4.2 K-Means Cluster Summary

| Cluster | Theme              | Example Movies                                |
|---------|--------------------|-----------------------------------------------|
| 0       | Dark Thrillers     | Parasite, Get Out, IT, A Quiet Place          |
| 1       | Family / Adventure | Toy Story, Harry Potter, Up, Coco             |
| 2       | Sci-Fi / Survival  | Inception, Interstellar, The Martian, Gravity |
| 3       | Crime / Drama      | The Godfather, Pulp Fiction, Joker            |
| 4       | Comedy             | The Hangover, Home Alone, Rush Hour           |
| 5       | Action / War       | The Dark Knight, 1917, Gladiator              |

The 6 clusters discovered by K-Means align very well with human intuition about movie genres. This confirms that the content-based feature representation is semantically meaningful.

## 5. Technical Details

| Component            | Details                                             |
|----------------------|-----------------------------------------------------|
| Vectorizer           | TF-IDF with ngram_range=(1,2), stop_words='english' |
| TF-IDF Matrix Shape  | 51 movies x 1096 vocabulary terms                   |
| Similarity Metric    | Cosine Similarity (sklearn.metrics.pairwise)        |
| Clustering Algorithm | K-Means, k=6, random_state=42, n_init=10            |
| Programming Language | Python 3.x                                          |
| Key Libraries        | scikit-learn, pandas, numpy, matplotlib, seaborn    |

## 6. Challenges Faced

- **Low similarity scores:** Cosine similarity scores are generally low (0.10 – 0.35) because the dataset is small and the combined features are short. With a larger dataset and richer descriptions, scores would be higher and more discriminative.
- **Choosing the right k for K-Means:** Selecting k=6 was done based on domain knowledge (roughly matching major genre categories). Ideally, the Elbow Method would be used to determine the optimal k objectively.
- **Cold start problem:** Content-based filtering cannot recommend movies to a new user with no history, and cannot recommend movies not in the database. This is a known limitation of content-based systems.
- **Building the dataset manually:** Curating 51 movies with accurate genre, director, keywords, and description fields required significant effort. Automation with a movie API like TMDB would streamline this in future.

## 7. What I Learned

- How TF-IDF converts raw text into numerical vectors and why inverse document frequency helps distinguish unique features
- How Cosine Similarity measures text similarity geometrically as the angle between feature vectors
- How recommendation systems work fundamentally and the difference between content-based and collaborative filtering approaches
- How K-Means clustering discovers structure in data without any labels — pure unsupervised learning in action
- How to structure a full ML project from dataset creation to evaluation and visualisation

## 8. Conclusion

The Movie Recommendation System successfully demonstrates content-based filtering using TF-IDF Vectorization and Cosine Similarity. The system produces logically consistent recommendations — Christopher Nolan films are grouped together, Jordan Peele thrillers appear near each other, and animated family films cluster naturally. The K-Means clustering layer further validates the quality of the feature representation.

While the system is constrained by its small, manually created dataset, it clearly demonstrates the core concepts of NLP-based feature extraction, similarity-based retrieval, and unsupervised clustering. Future extensions could include integrating the TMDB API for a live dataset, adding collaborative filtering based on user ratings, or building a Streamlit web interface for interactive recommendations.

---

Submitted by: ANURAG TIWARY | Course: Fundamentals of AI and ML